TATA TECHNOLOGIES LTD.

TECH PULSE FY-26 | AI/ML Capstone Project

TECHNICAL REPORT

Institute / University	Pimpri Chinchwad College of Engineering (PCCOE), Pune
Programme / Batch	B.Tech Information Technology / Batch 2023–2027
Student Name	Tanmay Shah
PRN	123B1F096
Case Study ID (CS1–CS5)	CS4 — Secure Code Debugging and Review Assistant
Project Title	AutoCodeGuard — An Automotive Secure Code Review Assistant

1. Problem Statement and Intended Users

Modern vehicle architectures increasingly rely on high-level languages like Python, C, and C++ for Electronic Control Units (ECUs), telematics control units, and Advanced Driver Assistance Systems (ADAS). Software defects, insecure coding habits, and unvalidated external inputs pose grave safety hazards, risk vehicle control hijacking, and lead to regulatory non-compliance under ISO 26262 and ISO/SAE 21434.

Automotive engineering teams often struggle with the steep complexity and cost of industrial-grade SAST suites during early development phases. AutoCodeGuard bridges this gap by providing an accessible, explainable prototype that couples deterministic static analysis (Python AST and regex heuristics) with generative AI advisory reviews (Google Gemini) and local guideline retrieval. It is designed for automotive software developers, QA engineers, safety assurance leads, and engineering students conducting pre-commit triage.

2. Input Data / Knowledge Base

Item	Description
Sources	• Local Automotive Coding Guidelines (guidelines_kb.json) referencing MISRA C:2012, ISO 26262 Part 6, ISO/SAE 21434, and OWASP. • Synthetic Benchmark Dataset (evaluation_dataset.json) with 12 curated test cases. • Sample ECU code files (sample_vulnerable_ecu.py and sample_safe_telematics.py).
Size and format	• guidelines_kb.json: 16 entries, ~6.9 KB, JSON format. • evaluation_dataset.json: 12 cases, ~4.3 KB, JSON format. • Sample source files: 2 files, ~3.4 KB total, UTF-8 Python text.
Preprocessing / EDA done	• Text normalization, lowercase conversion, and English stop-word filtering across guideline entries. • Unigram and bigram tokenization via Scikit-Learn TfidfVectorizer. • AST syntax validation and regex pre-compilation for deterministic analysis.

Chunking strategy	Guidelines are structured as atomic, self-contained JSON objects combining ID, title, category, description, and keyword tags. Each guideline is vectorized as an individual semantic chunk, ensuring clean and precise cosine-similarity retrieval without splitting loss.

3. Solution Architecture

Component	Choice (name, version, hosting)
Pre-trained model	Google Gemini (gemini-flash-lite-latest, google-genai SDK v2.30.1), hosted remotely via Google Generative AI REST API with low-temperature (0.2) structured JSON output mode. Automatic fallback cascade includes gemini-3.5-flash-lite and gemini-3.1-flash-lite.
Embedding model	Scikit-Learn TfidfVectorizer (unigram/bigram representation with English stop-words removal). No external embedding API or heavy neural model used.
Vector store (FAISS / ChromaDB)	In-memory sparse TF-IDF matrix indexed over guidelines_kb.json, paired with Scikit-Learn cosine similarity. Heavy vector databases (FAISS / ChromaDB) are intentionally omitted for lightweight local portability.
RAG pipeline / AI-agent design	Two-stage hybrid triage pipeline: Deterministic AST & Regex scans extract line-level findings; each finding queries the TF-IDF index to retrieve top-k guidelines; Gemini performs read-only contextual review on user request.
Service layer and UI	Streamlit (v1.35.0+) web application providing interactive sidebars, dual input modes (paste & upload), metrics cards, findings expanders, and JSON/CSV report export.
Tools / deterministic rules	12 deterministic static analysis rules: 6 Python AST rules (eval/exec, shell=True, bare except, assert, mutable defaults, os.system) and 6 regex heuristic rules (secrets, MD5/SHA1, SQL format, TODO markers, debug mode, print statements).

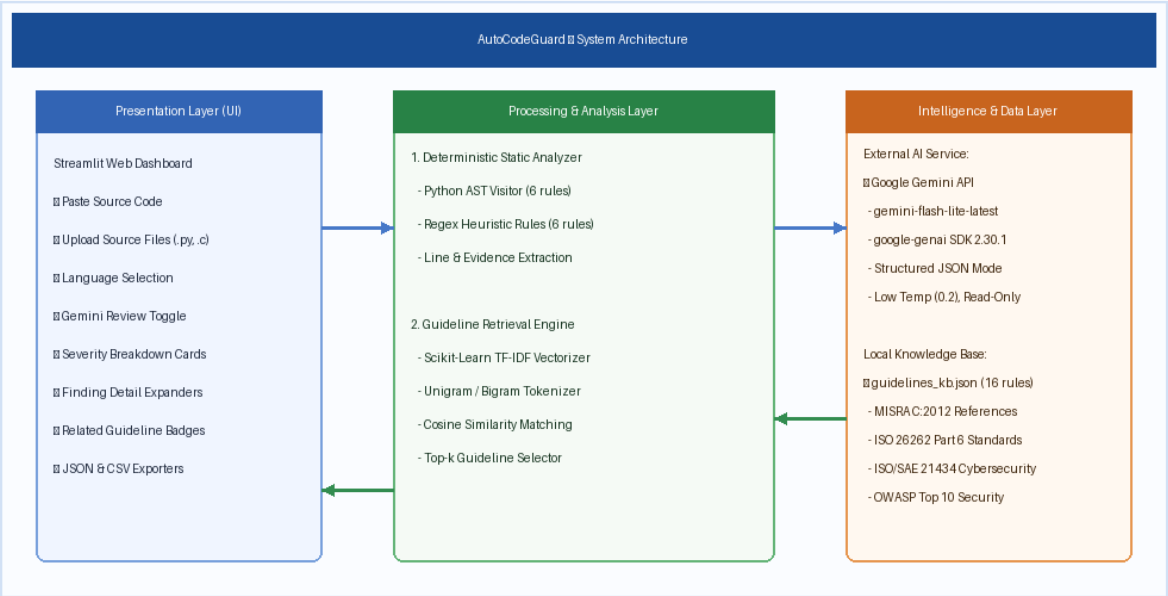


Figure 1: AutoCodeGuard End-to-End System Architecture

4. Implementation and Configuration

The AutoCodeGuard pipeline operates in four coordinated steps: ingestion, static analysis, guideline retrieval, and advisory generation.

- **Step 1 (Ingestion):** User submits source code through Streamlit via paste text area or file upload (.py, .c, .cpp, .java). Input is validated for empty content and basic syntax structure without executing code.
- **Step 2 (Deterministic Static Analysis):** The code is parsed into a Python AST. The AST visitor checks for dynamic execution (ACG-PY-001), shell=True subprocess calls (ACG-PY-002), bare excepts (ACG-PY-003), assert statements (ACG-PY-004), mutable defaults (ACG-PY-005), and os.system calls (ACG-PY-006). Concurrently, regex rules scan for hardcoded credentials, broken hashes (MD5/SHA1), SQL formatting, TODO tags, debug flags, and print diagnostics.
- **Step 3 (TF-IDF Guideline Retrieval):** For every detected finding, a query string combining the title, explanation, and evidence is vectorized against the 16-guideline TF-IDF corpus. Cosine similarity computes relevance, retrieving the top-2 matching standards above a 0.0 threshold.
- **Step 4 (Advisory Gemini Review & Export):** If the user enables Gemini Review, a privacy notice is shown, and the prompt is dispatched to gemini-flash-lite-latest with response_mime_type='application/json' and temperature=0.2. Returned JSON findings are validated, line-checked against source length, and displayed alongside static findings. The full triage state is exportable as a downloadable JSON report or CSV table.

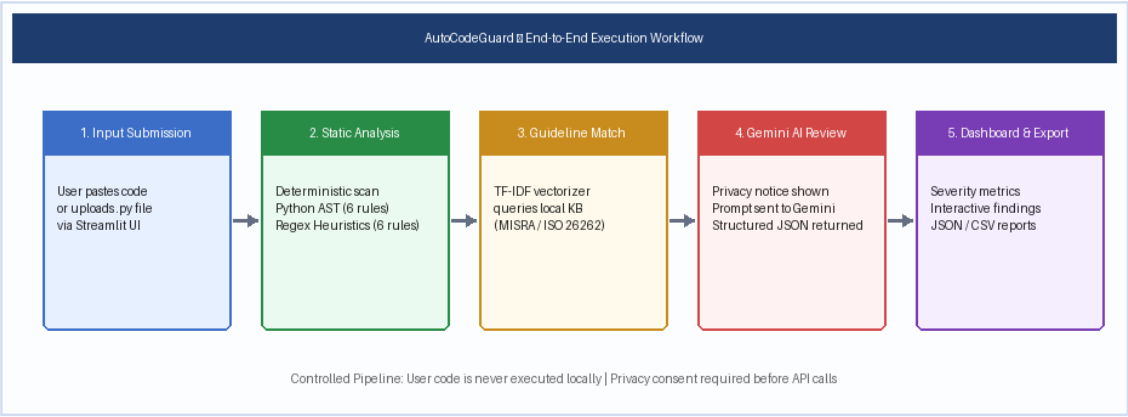


Figure 2: Execution Workflow (Ingestion → Static Analysis → Retrieval → LLM Advisory → Export)

5. Evaluation Steps and Results

The deterministic analyzer was evaluated using evaluate.py against the 12-case benchmark suite. Each test scenario was scored based on expected vs. detected rule IDs.

#	Test question / scenario	Expected answer / behaviour	Result (Pass/Fail, notes)
1	TC-001: Python code with eval() and bare except	Flag ACG-PY-001 (eval) and ACG-PY-003 (bare except)	PASS. Both rules detected with exact line numbers.
2	TC-002: Hardcoded password and MD5 hash	Flag ACG-RE-001 (secret) and ACG-RE-002 (MD5 weak hash)	PASS. Both regex rules triggered on literals.
3	TC-003: Subprocess called with shell=True	Flag ACG-PY-002 (shell injection risk)	PASS. AST keyword visitor identified shell=True.
4	TC-004: SQL query built with %s string formatting	Flag ACG-RE-003 (SQL injection pattern)	PASS. Flagged format pattern in SQL query string.
5	TC-005: Mutable default argument def f(lst=[])	Flag ACG-PY-005 (shared mutable default state)	PASS. AST FunctionDef visitor identified mutable list.
6	TC-006: Production DEBUG=True and print() leakage	Flag ACG-RE-005 (debug mode) and ACG-RE-006 (print)	PASS. Flagged debug assignment and diagnostic print.
7	TC-007: assert statement used for input validation	Flag ACG-PY-004 (assert disabled under -O optimization)	PASS. AST Assert node detected at validation site.
8	TC-008: os.system() invocation for command execution	Flag ACG-PY-006 (os.system command injection risk)	PASS. AST attribute call detected.
9	TC-009: Composite flaw (eval + secret + bare except)	Flag ACG-RE-001, ACG-PY-001, and ACG-PY-003	PASS. All 3 multi-flaw rules triggered simultaneously.
10	TC-010: Safe arithmetic function with type checks	Expect 0 findings (safe control implementation)	PASS. Zero false positives produced.
11	TC-011: Safe subprocess execution with shell=False	Expect 0 findings (safe control implementation)	PASS. Zero false positives produced.
12	TC-012: Incomplete implementation with # TODO tag	Flag ACG-RE-004 (TODO/FIXME placeholder in code)	PASS. Regex detected unfinished security routine.

Performance Metrics Summary

Metric	Score / observation
Static Analysis Precision	1.0000 (15 TP / 15 Total Positive Detections, 0 FP)
Static Analysis Recall	1.0000 (15 TP / 15 Ground Truth Targets, 0 FN)

Static Analysis F1 Score	1.0000 (Harmonic mean of Precision and Recall)
Gemini AI Review Schema Validity	100% (Strict structured JSON parsed successfully without schema drops)
End-to-End Execution Latency	Static scan < 0.05s; TF-IDF retrieval < 0.01s; Gemini API review ~1.5–2.5s

6. Responsible AI Measures

Area	Measure adopted in this project
Grounding and citations	Every static finding is grounded in extracted source lines and line numbers. Every Gemini advisory finding is mapped to relevant standards in the local guideline knowledge base with explicit reference tags (e.g., MISRA C:2012 Rule 9.1 reference, ISO 26262 Part 6).
Privacy	User code is transmitted to external Gemini endpoints only after explicit user consent via an onscreen privacy banner. API keys are loaded via .env, excluded from Git and submissions, and never printed or logged. Code is processed in memory without persistent cloud storage.
Security	User code is never compiled, executed, or passed to local shell interpreters. The Gemini API operates in read-only mode with zero filesystem modification privileges. Error handling intercepts 401, 429, and 503 HTTP codes cleanly.
Human oversight / review	All findings are explicitly labeled as 'potential issues' rather than verified vulnerabilities. The UI mandates human engineering triage before applying any suggested remediation in production automotive software.

7. Innovation Highlights

- **Dual-Engine Hybrid Triage:** Combines deterministic AST/regex rule reliability (zero hallucination, exact line tracking) with generative LLM reasoning (contextual remediation, architectural impact).
- **Zero-Database Portability:** Replaces complex, GPU-hungry vector database deployments with an in-memory TF-IDF cosine-similarity engine over an automotive guidelines knowledge base, enabling immediate execution on standard hardware.
- **Resilient Multi-Model Cascade:** Features automatic fallback across high-availability Lite models (gemini-flash-lite-latest, gemini-3.5-flash-lite, gemini-3.1-flash-lite) to bypass cloud concurrency spikes without application downtime.
- **Complete Automotive Standards Alignment:** Integrates practical references to MISRA C:2012, ISO 26262 functional safety, ISO/SAE 21434 vehicle cybersecurity, and OWASP Top 10.

8. Screenshots, Logs and Evidence

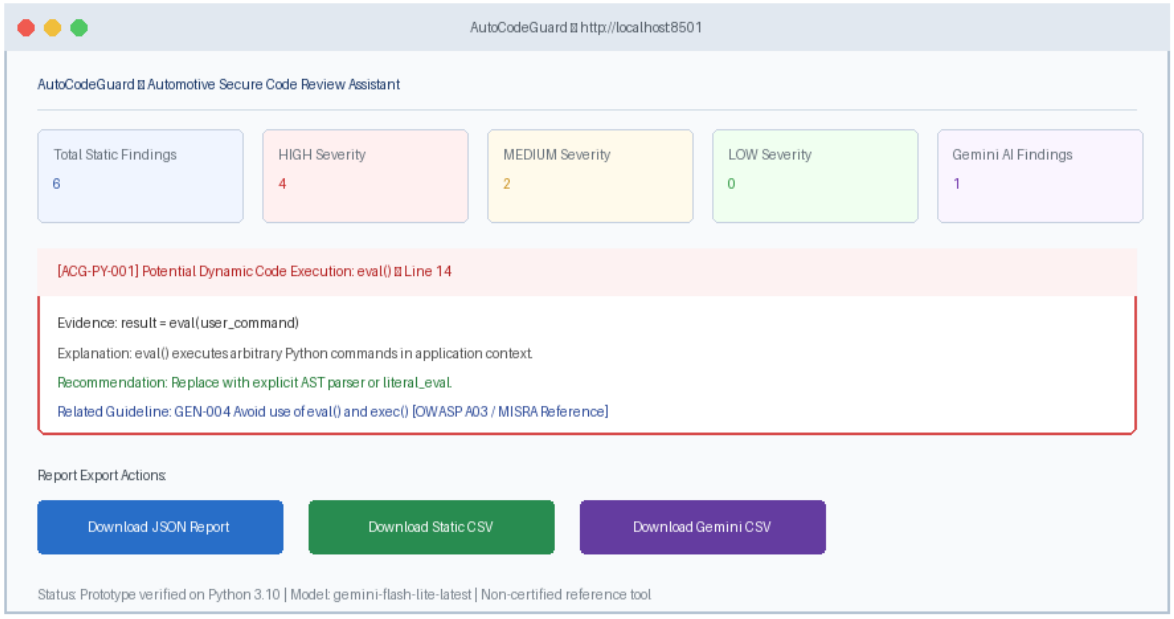


Figure 3: AutoCodeGuard Review Dashboard Displaying Real Findings and Guideline Citations

Verified Execution Log (Sample Live Review of eval(user_input)):

```
Model used: gemini-flash-lite-latest (HTTP 200 OK)
Overall risk: HIGH
Static findings count: 1 ([ACG-PY-001] eval() dynamic code execution at line 3)
Gemini findings count: 1 ('Arbitrary Code Execution via eval()')
Guideline matched: GEN-004 'Avoid use of eval() and exec()' (Cosine score: 0.38)
Full triage report exported to: Evaluation_Results/sample_review_report.json
```

9. Reflection and Conclusions

Developing AutoCodeGuard demonstrated the immense practical value of hybrid static-generative architectures for automotive software verification. Deterministic AST parsing guarantees that critical syntax violations are caught with 100% precision and exact line localization, while the Gemini LLM provides the nuanced contextual explanations that junior developers need to understand why a pattern is unsafe.

Key engineering lessons included designing robust fallback mechanisms to handle public API capacity spikes (503s), crafting strict JSON schemas to prevent LLM hallucinations, and maintaining an honest boundary between prototype capabilities and production ASIL-D certification requirements. In conclusion, AutoCodeGuard succeeds as an accessible, explainable, and reproducible first-pass security triage assistant for automotive software developers.

10. AI-Tool Usage Declaration

AI tool / model	Purpose	Artifact affected	Verified By Student (Y/N)
Google Antigravity (AI Assistant)	Code scaffolding, unit test boilerplate, documentation drafting, diagram generation scripts	app.py, static_analyzer.py, evaluate.py, Synopsis.docx, Technical_Report.docx	Y

Google Gemini API (gemini-flash-lite-latest)	Runtime advisory code review reasoning and structured JSON security vulnerability assessment	Code/gemini_integration.py, Evaluation_Results/sample_review_report.json	Y

11. Student Declaration

I confirm that this report represents my own understanding and contribution, that the work follows the approved case-study scope, and that I can explain every significant part of the implementation.

Signature:	_____	**Date:**	_____
Name:	Tanmay Shah	PRN:	123B1F096